

RDD TRANSFORMATIONS

Why This Matters

Transformations matter. They define how data is reshaped, aggregated, and prepared for analysis. Being able to understand how these different implications work helps write efficient spark applications at scale

Transformation Fundamentals

Transformations are lazy operations that define a computation on an RDD without executing it. They create a new RDD from an existing one.

```
# Each transformation returns a new RDD
```

```
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

```
mapped = rdd.map(lambda x: x * 2)
```

```
filtered = mapped.filter(lambda x: x > 5)
```

```
# Nothing has executed yet!
```

```
result = filtered.collect()
```

New, RDD, Not Executed

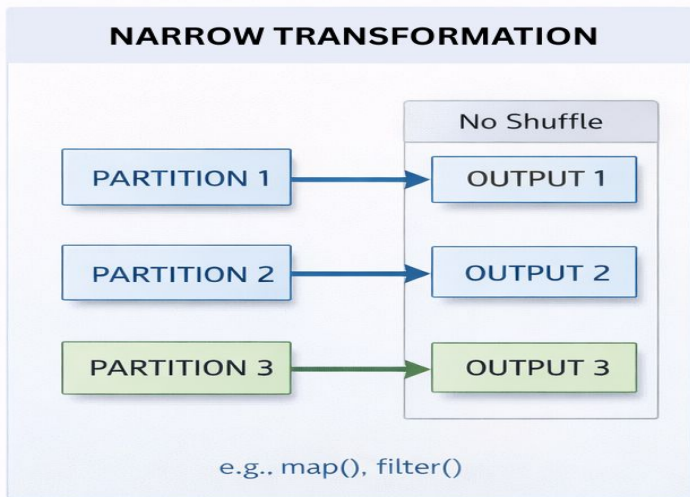
Another New RDD

NOW, the execution

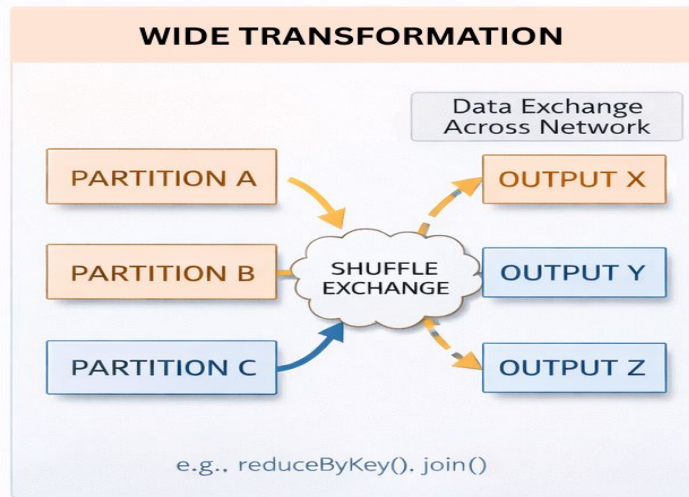
occurs

Narrow vs Wide Transformations

Narrow Transformations vs Wide Transformations

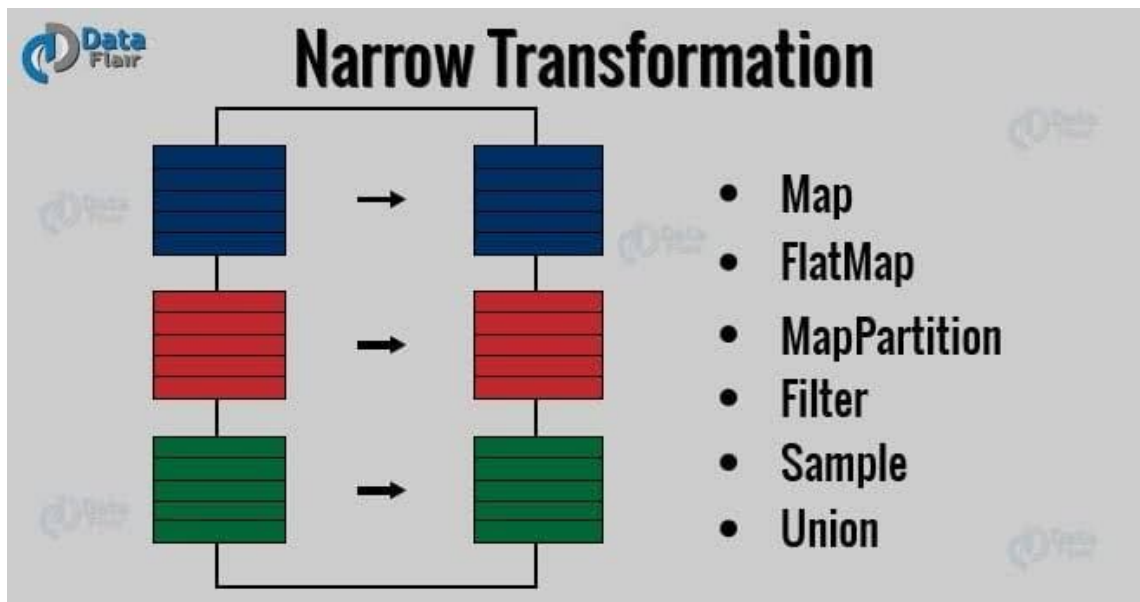


Each output partition maps to one input partition. For example, a filter processes partitions independently, producing one output partition per input.



Each output partition depends on multiple input partitions. For example, a join or groupBy reshuffles data, combining partitions to produce each output partition.

Narrow

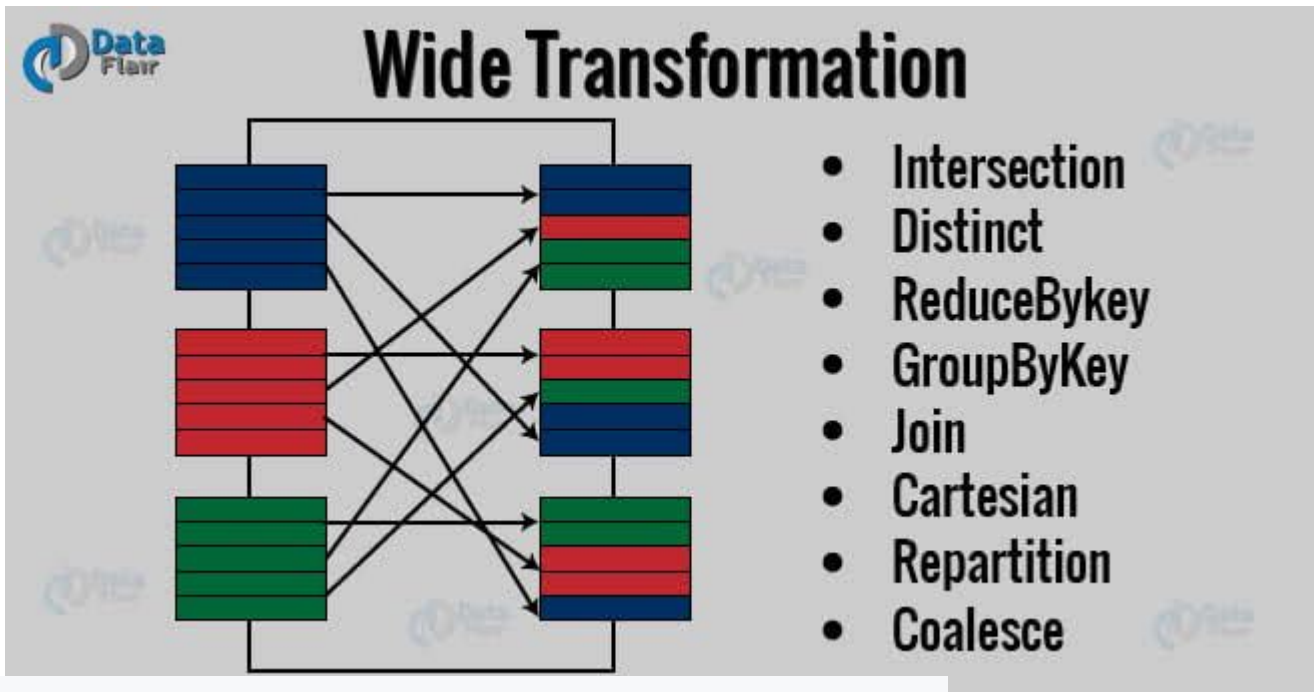


Parent Partition		Child Partition	
[1, 2]	->	[2, 4]	(map: $x * 2$)
[3, 4]	->	[6, 8]	
[5, 6]	->	[10, 12]	

Some Narrow Operations

Transformation	Description	Example
<code>map(func)</code>	Apply function to each element	<code>rdd.map(lambda x: x*2)</code>
<code>filter(func)</code>	Keep elements where func returns True	<code>rdd.filter(lambda x: x>0)</code>
<code>flatMap(func)</code>	Map + flatten results	<code>rdd.flatMap(lambda x: x.split())</code>

Wide



Parent Partitions

[("a",1),("b",2)]

Shuffle

Child Partitions

[("a",1),("a",3)]

[("a",3),("c",4)]

[("b",2),("c",4)]

WIDE Operations

Transformation	Description	Use Case
<code>mapPartitions(func)</code>	Apply function to entire partition	Database connections
<code>mapPartitionsWithIndex(func)</code>	Include partition index	Debugging
<code>coalesce(n)</code>	Reduce partitions (no shuffle if decreasing)	Reduce output files
<code>repartition(n)</code>	Increase/decrease partitions (shuffle)	Rebalance data